EXP-2

Python

```
!pip install requests textblob pandas


import requests

from textblob import TextBlob

import pandas as pd


# AIM-To Collect data from Social Media Platform/s using tools

video_id = "QJI0an6irrA"

api_key = "AIzaSyBLWVPXLR9R0ff3SwOL4c5BcvQrOcgU46s"


# Retrieve video information

video_info_url =
f"https://www.googleapis.com/youtube/v3/videos?part=snippet&id={video_id}&key={api_key}"

video_info_response = requests.get(video_info_url)

video_info_data = video_info_response.json()


print("Video Information Data:")

print(video_info_data)


# Retrieve video comments (Corrected URL format)

comments_url =
f"https://www.googleapis.com/youtube/v3/commentThreads?part=snippet&videoId={video_id}&key={api_key}"

comments_response = requests.get(comments_url)

comments_data = comments_response.json()


print("\nComments Data:")

print(comments_data)


# Extract the comments
```

```
comments = [
    item["snippet"]["topLevelComment"]["snippet"]["textOriginal"]
    for item in comments_data.get("items", [])
]
print(comments)

# -------- Analysis of YouTube Comments --------
def get_comment_sentiment(comment):
    analysis = TextBlob(comment)
    if analysis.sentiment.polarity > 0:
        return "Positive"
    elif analysis.sentiment.polarity == 0:
        return "Neutral"
    else:
        return "Negative"

# -------- Display the sentiment of YouTube video comments --------
comment_list = []
sentiment_list = []
for comment in comments:
    sentiment = get_comment_sentiment(comment)
    comment_list.append(comment)
    sentiment_list.append(sentiment)
    print(f"{comment} : {sentiment}")

sentiment_df = pd.DataFrame({
    "Comments": comment_list,
    "Sentiment": sentiment_list
})
print(sentiment_df)
sentiment_df.to_csv("YouTube_Comments_Sentiment.csv", index=False)
```

EXP-3

Python

```
!pip install pandas numpy matplotlib seaborn nltk textblob wordcloud plotly cufflinks scikit-learn

# Aim: Data Cleaning and Storage- Preprocess, filter and store social media data for business
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
import string
import re
import textblob
from textblob import TextBlob
import os
from wordcloud import WordCloud, STOPWORDS, ImageColorGenerator
import warnings
from google.colab import files
import plotly.graph_objects as go
from sklearn.feature_extraction.text import CountVectorizer
import cufflinks as cf

nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.stem.porter import PorterStemmer

%matplotlib inline
warnings.filterwarnings("ignore")
```

```
# Upload file
uploaded = files.upload()

# Assuming the uploaded file is 'train.csv'
tweets_df = pd.read_csv('train.csv')
print(tweets_df.head(5))
tweets_df.to_csv("airlinetweets.csv", index=False)

print(tweets_df.shape)
print(tweets_df.head())
tweets_df.info()
print(tweets_df['negativereason'].value_counts())

# Heat Map for missing Values
plt.figure(figsize=(17,5))
sns.heatmap(tweets_df.isnull(), cbar=True, yticklabels=False)
plt.xlabel("Column_Name", size=14, weight="bold")
plt.title("Places of missing values in column", size=17)
plt.show()

Top_Location_Of_tweet = tweets_df['airline'].value_counts().head(10)
print(Top_Location_Of_tweet)

# Using a set for 'stop' words for faster lookup and ensuring they are lowercase
stop = set(stopwords.words('english'))

# Convert the 'text' column to string type and fill NaN values with empty strings
tweets_df['text'] = tweets_df['text'].fillna('').astype(str)

# Process the text: split into words, remove stopwords, and join back to a string
tweets_df['text'] = tweets_df['text'].apply(
```

```
    lambda x: ' '.join([word for word in x.split() if word.lower() not in stop])
)
print(tweets_df.shape)

# Remove unnecessary characters
punct = ['[',']','/',',',',','.','"","""', '&',':','?']
def remove_punctuations(text):
    for punctuation in punct:
        text = text.replace(punctuation,'')
    return text

tweets_df['text'] = tweets_df['text'].apply(lambda x: remove_punctuations(x))

# Drop tweets that has empty text fields
tweets_df['text'].replace('', np.nan, inplace=True)
tweets_df.dropna(subset=['text'], inplace=True)
print(len(tweets_df))
tweets_df = tweets_df.reset_index(drop=True)
tweets_df.head()

sns.set_style('whitegrid')

# Use .update() method to add elements from a list to a set
stop.update(['Virgin America','San Francisco','Boston',
             'New York','customer','flight',
             'airline','San Diego','Oakland',
             'California'])

def plot_20_most_common_words(count_data, count_vectorizer):
    words = count_vectorizer.get_feature_names_out()
    total_counts = np.zeros(len(words))
```

```
    for t in count_data:
        total_counts += t.toarray()[0]
    count_dict = zip(words, total_counts)
    count_dict = sorted(count_dict, key=lambda x:x[1], reverse=True)[0:20]
    words = [w[0] for w in count_dict]
    counts = [w[1] for w in count_dict]
    x_pos = np.arange(len(words))
    plt.figure(figsize=(15,10))
    plt.subplot(title='20 most common words')
    sns.set_context('notebook', font_scale=1.5, rc={'lines.linewidth':2.5})
    sns.barplot(x=x_pos, y=counts, palette='husl')
    plt.xticks(x_pos, words, rotation=90)
    plt.xlabel('words')
    plt.ylabel('counts')
    plt.show()

count_vectorizer = CountVectorizer(stop_words=list(stop))
count_data = count_vectorizer.fit_transform(tweets_df['text'])

# Visualise the 20 most common words
plot_20_most_common_words(count_data, count_vectorizer)
plt.savefig('saved_figure.png')

cf.go_offline()
cf.set_config_file(offline=False, world_readable=True)

def get_top_n_bigram(corpus, n=None):
    vec = CountVectorizer(ngram_range=(2,2), stop_words='english').fit(corpus)
    bag_of_words = vec.transform(corpus)
    sum_words = bag_of_words.sum(axis=0)
    words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.items()]
```

```
    words_freq = sorted(words_freq, key=lambda x:x[1], reverse=True)
    return words_freq[:n]

common_words = get_top_n_bigram(tweets_df['text'], 8)
for word, freq in common_words:
    print(word, freq)

bigram_df = pd.DataFrame(common_words, columns=['ngram', 'count'])
bigram_df.groupby('ngram').sum()['count'].sort_values().plot.barh(
    title='Top 8 bigrams',
    color='orange',
    width=.4,
    figsize=(12,8),
    stacked=True
)
plt.show()

def get_subjectivity(text):
    return TextBlob(text).sentiment.subjectivity

def get_polarity(text):
    return TextBlob(text).sentiment.polarity

tweets_df['subjectivity'] = tweets_df['text'].apply(get_subjectivity)
tweets_df['polarity'] = tweets_df['text'].apply(get_polarity)
print(tweets_df.head())
```

EXERCISE-1

Python

```
# Install library
!pip install google-api-python-client pandas matplotlib seaborn wordcloud
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from googleapiclient.discovery import build
from wordcloud import WordCloud

# Enter API Key
api_key = "AIzaSyBLWVPXLR9R0ff3SwOL4c5BcvQrOcgU46s"
youtube = build('youtube', 'v3', developerKey=api_key)

# Video ID
video_id = "QJI0an6irrA"

# Fetch comments
request = youtube.commentThreads().list(
    part="snippet",
    videoId=video_id,
    maxResults=100
)
response = request.execute()

comments = []
for item in response.get('items', []):
    comment = item['snippet']['topLevelComment']['snippet']['textDisplay']
    author = item['snippet']['topLevelComment']['snippet']['authorDisplayName']
    likes = item['snippet']['topLevelComment']['snippet']['likeCount']
    comments.append([author, comment, likes])

# Store in CSV
df = pd.DataFrame(comments, columns=['User', 'Comment', 'Likes'])
df.to_csv("youtube_comments.csv", index=False)
```

```
print(df.head())

# Remove null values
df.dropna(inplace=True)

# Convert to lowercase
df['Comment'] = df['Comment'].str.lower()

# Top liked comments
top = df.sort_values(by='Likes', ascending=False).head(10)
plt.figure(figsize=(10,6))
sns.barplot(x='Likes', y='User', data=top)
plt.title("Top Liked Comments")
plt.show()

# WordCloud
text = " ".join(i for i in df['Comment'])
wordcloud = WordCloud(width=500, height=400, background_color='white').generate(text)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()
```

EXERCISE-2

Python

```
!pip install pandas matplotlib seaborn textblob wordcloud

# ========================
# TWITTER DATA ANALYSIS PROJECT
# ========================
import pandas as pd
import re
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from textblob import TextBlob
from collections import Counter
from wordcloud import WordCloud

# ========================
# STEP 1: CREATE SAMPLE TWITTER DATA
# ========================
data = {
    "date": [
        "2024-01-01","2024-01-02","2024-01-03","2024-01-04","2024-01-05",
        "2024-01-06","2024-01-07","2024-01-08","2024-01-09","2024-01-10"
    ],
    "user": [
        "u1","u2","u3","u4","u5","u6","u7","u8","u9","u10"
    ],
    "text": [
        "AI is the future #AI #Tech",
        "Loving new AI tools #Innovation",
        "AI will replace jobs #Concern",
        "Amazing progress in AI #Tech",
        "AI is dangerous #Risk",
        "AI helps businesses grow #AI",
        "New trends in machine learning #ML",
        "AI innovation is fast #Innovation",
        "Worried about AI impact #Concern",
        "AI is transforming industries #Tech"
    ]
}
df = pd.DataFrame(data)
```

```
print("\nDataset:\n", df)

# ========================
# STEP 2: TEXT PREPROCESSING
# ========================
def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)  # remove links
    text = re.sub(r'[^a-z#\s]', '', text)       # keep hashtags
    return text
df['clean_text'] = df['text'].apply(clean_text)

# ========================
# STEP 3: SENTIMENT ANALYSIS
# ========================
def get_sentiment(text):
    score = TextBlob(text).sentiment.polarity
    if score > 0:
        return "Positive"
    elif score < 0:
        return "Negative"
    else:
        return "Neutral"
df['sentiment'] = df['clean_text'].apply(get_sentiment)
print("\nSentiment Count:\n", df['sentiment'].value_counts())

# ========================
# STEP 4: HASHTAG ANALYSIS
# ========================
def extract_hashtags(text):
    return re.findall(r"#\w+", text)
```

```
df['hashtags'] = df['clean_text'].apply(extract_hashtags)
all_tags = [tag for tags in df['hashtags'] for tag in tags]
tag_counts = Counter(all_tags)
print("\nTop Hashtags:")
for tag, count in tag_counts.most_common(5):
    print(tag, ":", count)

# ========================
# STEP 5: WORD FREQUENCY
# ========================
all_words = " ".join(df['clean_text']).split()
stopwords = {"the","is","and","to","of","in","on","for","about"}
filtered_words = [w for w in all_words if w not in stopwords]
word_counts = Counter(filtered_words)
print("\nTop Keywords:")
for word, count in word_counts.most_common(5):
    print(word, ":", count)

# ========================
# STEP 6: VISUALIZATIONS
# ========================
# Sentiment Chart
plt.figure(figsize=(6,4))
sns.countplot(x='sentiment', data=df)
plt.title("Sentiment Distribution")
plt.savefig("sentiment.png")
plt.show()

# Hashtag Chart
top_tags = dict(tag_counts.most_common(5))
plt.figure(figsize=(6,4))
```

```
plt.bar(top_tags.keys(), top_tags.values())

plt.title("Top Hashtags")

plt.savefig("hashtags.png")

plt.show()


# WordCloud

wordcloud = WordCloud(width=800, height=400, background_color='white').generate(" ".join(filtered_words))

plt.figure(figsize=(8,4))

plt.imshow(wordcloud, interpolation='bilinear')

plt.axis("off")

plt.title("Word Cloud")

plt.savefig("wordcloud.png")

plt.show()


# ========================

# STEP 7: SAVE OUTPUT

# ========================

df.to_csv("processed_twitter_data.csv", index=False)

print("\nAnalysis Complete")
```

EXP-4

Python

```
# Install required libraries

!pip install google-api-python-client pandas matplotlib seaborn textblob wordcloud


import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from textblob import TextBlob

from wordcloud import WordCloud
```

```
import warnings
warnings.filterwarnings("ignore")
from googleapiclient.discovery import build

# YouTube API Key
api_key = "AIzaSyBLWVPXLR9R0ff3SwOL4c5BcvQrOcgU46s"
youtube = build("youtube", "v3", developerKey=api_key)

# Enter Video ID
video_id = "QJI0an6irrA"
request = youtube.commentThreads().list(
    part="snippet",
    videoId=video_id,
    maxResults=100
)
response = request.execute()
comments = []

for item in response.get("items", []):
    comment = item["snippet"]["topLevelComment"]["snippet"]["textDisplay"]
    comments.append(comment)

df = pd.DataFrame(comments, columns=["Comments"])
print(df.head())

# Save comments
df.to_csv("youtube_comments.csv", index=False)

# Data Cleaning
df.dropna(inplace=True)
df["Comments"] = df["Comments"].str.lower()
```

```
# Sentiment Analysis
def get_sentiment(text):
    score = TextBlob(text).sentiment.polarity
    if score > 0:
        return "Positive"
    elif score < 0:
        return "Negative"
    else:
        return "Neutral"

df["Sentiment"] = df["Comments"].apply(get_sentiment)
print(df.head())

# Count sentiments
print(df["Sentiment"].value_counts())

# Bar chart
plt.figure()
sns.countplot(x="Sentiment", data=df)
plt.title("Sentiment Analysis")
plt.show()

# WordCloud Positive
text_pos = " ".join(df[df["Sentiment"]=="Positive"]["Comments"])
if text_pos:
    wordcloud = WordCloud(width=800, height=400, background_color="white").generate(text_pos)
    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.axis("off")
    plt.title("Positive WordCloud")
```

```
    plt.show()

# WordCloud Negative
text_neg = " ".join(df[df["Sentiment"]=="Negative"]["Comments"])
if text_neg:
    wordcloud = WordCloud(width=800, height=400, background_color="white").generate(text_neg)
    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.axis("off")
    plt.title("Negative WordCloud")
    plt.show()
```

EXERCISE-1 (Under EXP-4)

Python

```
# Install libraries
!pip install google-api-python-client pandas textblob wordcloud matplotlib

import pandas as pd
from googleapiclient.discovery import build
from textblob import TextBlob
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# API Key
api_key = "AIzaSyBLWVPXLR9R0ff3SwOL4c5BcvQrOcgU46s"
youtube = build("youtube", "v3", developerKey=api_key)

# Enter Video ID
video_id = "QJI0an6irrA"

# Fetch Comments
request = youtube.commentThreads().list(
```

```
    part="snippet",
    videoId=video_id,
    maxResults=100
)
response = request.execute()
comments = []

for item in response.get('items', []):
    text = item['snippet']['topLevelComment']['snippet']['textDisplay']
    comments.append(text)

# DataFrame
df = pd.DataFrame(comments, columns=['Comment'])

# Save CSV
df.to_csv("youtube_comments_2.csv", index=False)

# Sentiment Function
def get_sentiment(text):
    score = TextBlob(text).sentiment.polarity
    if score > 0:
        return "Positive"
    elif score < 0:
        return "Negative"
    else:
        return "Neutral"
df['Sentiment'] = df['Comment'].apply(get_sentiment)
print(df.head())

for sentiment in ['Positive','Negative','Neutral']:
    text = " ".join(df[df['Sentiment']==sentiment]['Comment'])
```

```
    if text:
        wc = WordCloud(width=800, height=400, background_color='white').generate(text)
        plt.figure(figsize=(10,5))
        plt.imshow(wc, interpolation='bilinear')
        plt.title(sentiment + " WordCloud")
        plt.axis("off")
        plt.show()
```

EXERCISE-2 (Under EXP-4)

Python

```
!pip install pandas textblob matplotlib seaborn


import pandas as pd
import re
from textblob import TextBlob
import matplotlib.pyplot as plt
import seaborn as sns


# Example dataset (you can replace with Twitter/Instagram data)
data = {
    "Brand": ["Apple", "Apple", "Samsung", "Samsung", "Apple", "Samsung"],
    "Text": [
        "Amazing phone, love it!",
        "Battery is bad",
        "Great display quality",
        "Worst customer service",
        "Best performance ever",
        "Good value for money"
    ]
}
df = pd.DataFrame(data)
```

```
# Clean text function
def clean_text(text):
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^A-Za-z\s]", "", text)
    return text.lower()
df["Cleaned"] = df["Text"].apply(clean_text)

# Sentiment function
def get_sentiment(text):
    polarity = TextBlob(text).sentiment.polarity
    if polarity > 0:
        return "Positive"
    elif polarity < 0:
        return "Negative"
    else:
        return "Neutral"
df["Sentiment"] = df["Cleaned"].apply(get_sentiment)

# Count sentiments per brand
sentiment_counts = df.groupby(["Brand", "Sentiment"]).size().unstack().fillna(0)
print(sentiment_counts)

# Bar Chart
sentiment_counts.plot(kind="bar", figsize=(8,5))
plt.title("Comparative Sentiment Analysis")
plt.ylabel("Number of Comments")
plt.xticks(rotation=0)
plt.show()

# Pie Chart (per brand)
for brand in df["Brand"].unique():
```

```
    subset = df[df["Brand"] == brand]["Sentiment"].value_counts()
    plt.figure()
    subset.plot(kind="pie", autopct='%1.1f%%')
    plt.title(f"{brand} Sentiment Distribution")
    plt.ylabel("")
    plt.show()

plt.figure()
sns.countplot(x='Brand', hue='Sentiment', data=df)
plt.title("Brand Sentiment Comparison")
plt.xlabel("Brand")
plt.ylabel("Count")
plt.show()
```

EXP-5

Python

```
# =========================
# 1. Install required library
# =========================
!pip install networkx matplotlib

# =========================
# 2. Import libraries
# =========================
import networkx as nx
import matplotlib.pyplot as plt

# =========================
# 3. Create Social Media Graph
# =========================
G = nx.Graph()
# Sample users (nodes)
```

```
users = ["A", "B", "C", "D", "E", "F", "G", "H"]
G.add_nodes_from(users)
# Sample interactions (edges)
connections = [
    ("A", "B"), ("A", "C"), ("B", "C"),
    ("B", "D"), ("C", "D"),
    ("D", "E"), ("E", "F"),
    ("F", "G"), ("G", "H"),
    ("E", "H"), ("C", "F")
]
G.add_edges_from(connections)
print("Graph Created Successfully!")
print("Nodes:", G.nodes())
print("Edges:", G.edges())

# ========================
# 4. Centrality Measures
# ========================
degree_centrality = nx.degree_centrality(G)
closeness_centrality = nx.closeness_centrality(G)
betweenness_centrality = nx.betweenness_centrality(G)
eigenvector_centrality = nx.eigenvector_centrality(G)

print("\n--- Degree Centrality ---")
print(degree_centrality)
print("\n--- Closeness Centrality ---")
print(closeness_centrality)
print("\n--- Betweenness Centrality ---")
print(betweenness_centrality)
print("\n--- Eigenvector Centrality ---")
print(eigenvector_centrality)
```

```
# ========================
# 5. Community Detection
# ========================
from networkx.algorithms.community import greedy_modularity_communities
communities = list(greedy_modularity_communities(G))
print("\n--- Communities Detected ---")
for i, community in enumerate(communities):
    print(f"Community {i+1}: {list(community)}")

# ========================
# 6. Graph Visualization
# ========================
plt.figure(figsize=(8,6))
pos = nx.spring_layout(G, seed=42)
# Color nodes based on community
colors = []
for node in G.nodes():
    for i, community in enumerate(communities):
        if node in community:
            colors.append(i)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=colors,
    node_size=800,
    cmap=plt.cm.Set3,
    edge_color="gray"
)
```

```
plt.title("Social Media Network Graph")
plt.show()
```

EXERCISE-1 (Under EXP-5)

Python

```
# ========================
# 1. Install libraries
# ========================
!pip install networkx matplotlib

# ========================
# 2. Import libraries
# ========================
import networkx as nx
import matplotlib.pyplot as plt

# ========================
# 3. Create Graph
# ========================
G = nx.Graph()

# ========================
# 4. Nodes from diagram
# ========================
nodes = [
    "A","B","C","D","E","F","Q","O",
    "P","J","K","L","M","N","G","H","I"
]
G.add_nodes_from(nodes)

# ========================
# 5. Edges (from diagram structure)
```

```
# ========================
edges = [
    # Left cluster
    ("P","J"), ("P","G"), ("P","H"), ("P","I"),
    ("P","L"), ("P","M"), ("P","N"), ("P","K"),
    ("J","K"), ("K","L"), ("L","M"), ("M","N"),
    ("J","G"), ("G","H"), ("H","I"),
    # Bridge connection
    ("P","F"),
    # Right cluster
    ("F","Q"), ("F","O"), ("F","C"),
    ("Q","A"), ("Q","B"), ("Q","C"), ("Q","D"), ("Q","E"), ("Q","O"),
    ("A","E"), ("A","D"),
    ("B","C"), ("B","D"), ("B","E"),
    ("C","D"), ("C","O"),
    ("D","E"),
    ("E","F")
]
G.add_edges_from(edges)
print("Graph Created Successfully!")
print("Nodes:", len(G.nodes()))
print("Edges:", len(G.edges()))

# ========================
# 6. Centrality Measures
# ========================
degree_centrality = nx.degree_centrality(G)
eigenvector_centrality = nx.eigenvector_centrality(G)

# ========================
# 7. Top 3 Influential Users
```

```
# ========================
top_degree = sorted(degree_centrality.items(), key=lambda x: x[1], reverse=True)[:3]
top_eigen = sorted(eigenvector_centrality.items(), key=lambda x: x[1], reverse=True)[:3]

print("\nTop 3 by Degree Centrality:")
for node, val in top_degree:
    print(node, ":", round(val, 3))

print("\nTop 3 by Eigenvector Centrality:")
for node, val in top_eigen:
    print(node, ":", round(val, 3))

# ========================
# 8. Visualization
# ========================
plt.figure(figsize=(10,7))
pos = nx.spring_layout(G, seed=42)
# node color based on eigenvector importance
colors = [eigenvector_centrality[node] for node in G.nodes()]
nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=800,
    node_color=colors,
    cmap=plt.cm.plasma,
    edge_color="gray"
)
plt.title("Social Media Interaction Network")
plt.show()
```

EXERCISE-2 (Under EXP-5)

Python

```
# ========================
# 1. Install required libraries
# ========================
!pip install networkx matplotlib python-louvain

# ========================
# 2. Import libraries
# ========================
import networkx as nx
import matplotlib.pyplot as plt
import community as community_louvain

# ========================
# 3. Create Real-life Social Network
# ========================
G = nx.Graph()
# Users (nodes)
users = [
    "Alice","Bob","Charlie","David","Emma","Frank","Grace","Helen",
    "Ivy","Jack","Karan","Lina","Mona","Nina","Owen","Paul"
]
G.add_nodes_from(users)
# Interactions (edges)
edges = [
    # Group 1 (friends / influencers cluster)
    ("Alice","Bob"), ("Alice","Charlie"), ("Bob","Charlie"),
    ("Charlie","David"), ("David","Emma"), ("Emma","Frank"),
    # Group 2 (another community)
    ("Grace","Helen"), ("Helen","Ivy"), ("Ivy","Jack"),
    ("Jack","Grace"), ("Grace","Ivy"),
```

```
    # Group 3
    ("Karan","Lina"), ("Lina","Mona"), ("Mona","Nina"),
    ("Nina","Karan"),
    # Bridge connections
    ("Frank","Grace"),
    ("Emma","Helen"),
    ("Jack","Karan"),
    ("David","Ivy")
]
G.add_edges_from(edges)
print("Graph Created!")
print("Nodes:", G.number_of_nodes())
print("Edges:", G.number_of_edges())

# ========================
# 4. Apply Community Detection (Louvain)
# ========================
partition = community_louvain.best_partition(G)
# Print communities
communities = {}
for node, comm_id in partition.items():
    communities.setdefault(comm_id, []).append(node)
print("\n--- Detected Communities ---")
for comm_id, members in communities.items():
    print(f"Community {comm_id+1}: {members}")

# ========================
# 5. Analyze Connectivity
# ========================
print("\n--- Connectivity Analysis ---")
print("Density:", nx.density(G))
```

```
print("Average Clustering Coefficient:", nx.average_clustering(G))

# ========================
# 6. Identify Key Bridge Users
# ========================
betweenness = nx.betweenness_centrality(G)
top_bridge = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:3]
print("\n--- Top Bridge (Influential) Users ---")
for node, val in top_bridge:
    print(node, ":", round(val, 3))

# ========================
# 7. Visualization
# ========================
plt.figure(figsize=(10,7))
pos = nx.spring_layout(G, seed=42)
# Color nodes by community
colors = [partition[node] for node in G.nodes()]
nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=colors,
    node_size=800,
    cmap=plt.cm.Set2,
    edge_color="gray"
)
plt.title("Community Detection in Social Media Network")
plt.show()
```

EXP-7

Python

```
# ========================
# COMMUNALYTIC-LIKE ANALYSIS (VS CODE VERSION)
# ========================
# Install libraries
!pip install pandas matplotlib networkx textblob wordcloud

import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx
from textblob import TextBlob
from wordcloud import WordCloud

# ========================
# CREATE SAMPLE DATASET
# ========================
data = {
    "user": [
        "u1","u2","u3","u4","u5","u6","u7","u8","u9","u10"
    ],
    "text": [
        "Nike shoes are amazing",
        "I love Nike products",
        "Nike quality is great",
        "Too expensive but good",
        "Not satisfied with Nike",
        "Best sports brand ever",
        "Nike sneakers are stylish",
        "Bad experience with delivery",
        "Excellent performance shoes",
        "Worth the price"
    ],
```

```
    "mention": [
        "u2","u3","u4","u5","u6","u7","u8","u9","u10","u1"
    ]
}
df = pd.DataFrame(data)
print("\nDataset:\n", df)

# ========================
# TEXT ANALYSIS
# ========================
# Sentiment Analysis
df['sentiment'] = df['text'].apply(lambda x: TextBlob(str(x)).sentiment.polarity)
df['sentiment_type'] = df['sentiment'].apply(
    lambda x: "Positive" if x > 0 else ("Negative" if x < 0 else "Neutral")
)
print("\nSentiment Counts:\n", df['sentiment_type'].value_counts())

# ========================
# WORD CLOUD
# ========================
text_data = " ".join(df['text'])
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(text_data)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.title("Keyword Frequency")
plt.savefig("wordcloud_exp7.png")
plt.show()

# ========================
# SENTIMENT BAR CHART
```

```
# ========================
df['sentiment_type'].value_counts().plot(kind='bar')
plt.title("Sentiment Distribution")
plt.savefig("sentiment_chart_exp7.png")
plt.show()

# ========================
# NETWORK ANALYSIS
# ========================
G = nx.Graph()
for i, row in df.iterrows():
    G.add_edge(row['user'], row['mention'])

print("\nGraph Info:")
print("Nodes:", G.number_of_nodes())
print("Edges:", G.number_of_edges())

# ========================
# INFLUENCER DETECTION
# ========================
degree = nx.degree_centrality(G)
betweenness = nx.betweenness_centrality(G)
top_degree = sorted(degree.items(), key=lambda x: x[1], reverse=True)[:3]
top_between = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:3]

print("\nTop Degree Influencers:")
for u, v in top_degree:
    print(u, round(v,3))
print("\nTop Betweenness Influencers:")
for u, v in top_between:
    print(u, round(v,3))
```

```
# ========================
# NETWORK VISUALIZATION
# ========================
pos = nx.spring_layout(G, seed=42)
plt.figure(figsize=(8,6))
nx.draw(G, pos, with_labels=True, node_size=1000)
plt.title("Social Media Network")
plt.savefig("network_graph_exp7.png")
plt.show()
```

EXERCISE-1 (Under EXP-7)

Python

```
# Install libraries
!pip install pandas matplotlib wordcloud

# ========================
# EXERCISE 1:
# KEYWORD FREQUENCY ANALYSIS (VS CODE VERSION)
# ========================
import pandas as pd
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from collections import Counter
import re

# ========================
# CREATE SAMPLE DATASET
# ========================
data = {
    "text": [
        "Apple products are amazing and innovative",
```

```
        "I love the new iPhone from Apple",
        "Apple has great design and quality",
        "Battery life of Apple devices is impressive",
        "Apple is too expensive but worth it",
        "Customer service of Apple is excellent",
        "Not happy with Apple update",
        "Apple MacBook performance is outstanding",
        "Apple ecosystem is very smooth",
        "Some Apple products are overpriced"
    ]
}
df = pd.DataFrame(data)
print("\nDataset:\n", df)

# ========================
# TEXT PREPROCESSING
# ========================
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    return text
df['clean_text'] = df['text'].apply(clean_text)

# ========================
# KEYWORD FREQUENCY ANALYSIS
# ========================
all_words = " ".join(df['clean_text']).split()
# Stopwords
stopwords = set([
    "the","is","and","are","of","to","with","a","an","it","very","but"
])
```

```
filtered_words = [word for word in all_words if word not in stopwords]
word_counts = Counter(filtered_words)

print("\nTop Keywords:")
for word, count in word_counts.most_common(10):
    print(word, ":", count)

# ========================
# BAR CHART
# ========================
top_words = dict(word_counts.most_common(10))
plt.figure(figsize=(10,5))
plt.bar(top_words.keys(), top_words.values())
plt.xticks(rotation=45)
plt.title("Top Keywords Frequency")
plt.xlabel("Keywords")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("keyword_bar_chart.png")
plt.show()

# ========================
# WORD CLOUD
# ========================
wordcloud = WordCloud(
    width=800,
    height=400,
    background_color='white'
).generate(" ".join(filtered_words))
plt.figure(figsize=(10,5))
plt.imshow(wordcloud, interpolation='bilinear')
```

```
plt.axis("off")
plt.title("Keyword Word Cloud")
plt.savefig("wordcloud_ex1_exp7.png")
plt.show()
```

EXERCISE-2 (Under EXP-7)

Python

```
# Install libraries
!pip install networkx matplotlib python-louvain

# ========================
# EXERCISE 2:
# IDENTIFY INFLUENTIAL USERS & COMMUNITIES (VS CODE VERSION)
# ========================
import networkx as nx
import matplotlib.pyplot as plt
import community as community_louvain

# ========================
# CREATE SAMPLE SOCIAL NETWORK
# ========================
edges = [
    # Community 1
    ("A","B"), ("A","C"), ("B","C"), ("C","D"), ("D","E"), ("E","A"),
    # Community 2
    ("F","G"), ("F","H"), ("G","H"), ("H","I"), ("I","J"), ("J","F"),
    # Community 3
    ("K","L"), ("K","M"), ("L","M"), ("M","N"), ("N","O"), ("O","K"),
    # Bridge connections (important users)
    ("E","F"), ("D","H"), ("J","K")
]
G = nx.Graph()
```

```
G.add_edges_from(edges)
print("\nGraph Created!")
print("Nodes:", G.number_of_nodes())
print("Edges:", G.number_of_edges())

# ========================
# CENTRALITY (INFLUENCERS)
# ========================
degree = nx.degree_centrality(G)
betweenness = nx.betweenness_centrality(G)
eigenvector = nx.eigenvector_centrality(G)

# Top influencers
top_degree = sorted(degree.items(), key=lambda x: x[1], reverse=True)[:3]
top_between = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:3]
top_eigen = sorted(eigenvector.items(), key=lambda x: x[1], reverse=True)[:3]

print("\nTop Degree Influencers:")
for u, v in top_degree:
    print(u, round(v,3))
print("\nTop Betweenness Influencers (Bridges):")
for u, v in top_between:
    print(u, round(v,3))
print("\nTop Eigenvector Influencers:")
for u, v in top_eigen:
    print(u, round(v,3))

# ========================
# COMMUNITY DETECTION (LOUVAIN)
# ========================
partition = community_louvain.best_partition(G)
```

```
communities = {}
for node, comm_id in partition.items():
    communities.setdefault(comm_id, []).append(node)

print("\nDetected Communities:")
for cid, members in communities.items():
    print(f"Community {cid+1}: {members}")

# ========================
# NETWORK PROPERTIES
# ========================
print("\nNetwork Density:", round(nx.density(G), 3))
print("Average Clustering Coefficient:", round(nx.average_clustering(G), 3))

# ========================
# VISUALIZATION
# ========================
pos = nx.spring_layout(G, seed=42)
colors = [partition[node] for node in G.nodes()]
plt.figure(figsize=(10,7))
nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=1000,
    node_color=colors,
    cmap=plt.cm.Set2
)
plt.title("Community Detection & Influencer Network")
plt.savefig("community_network.png")
plt.show()
```

EXP-8

Python

```
!pip install pandas matplotlib seaborn geopy geopandas shapely fiona pyproj


# ========================
# COMPLETE SOCIAL MEDIA ANALYSIS (VS CODE)
# ========================
import pandas as pd
import re
import matplotlib.pyplot as plt
import seaborn as sns
from geopy.geocoders import Nominatim
import geopandas as gpd


# ========================
# PART 1: MENTION ANALYSIS
# ========================
data_mentions = {
    'Text': [
        "Great meeting with @user1 today!",
        "Excited to announce our partnership with @user2.",
        "Thanks for the shoutout @user3,@user2 and we appreciate it!",
        "Had a productive discussion with @user4 and @user5."
    ]
}
df_mentions = pd.DataFrame(data_mentions)


def extract_mentions(text):
    return re.findall(r'@(\w+)', text)


df_mentions['Mentions'] = df_mentions['Text'].apply(extract_mentions)
```

```
all_mentions = [m for sublist in df_mentions['Mentions'] for m in sublist]
mentions_df = pd.DataFrame(all_mentions, columns=['Mention'])
mention_counts = mentions_df['Mention'].value_counts().reset_index()
mention_counts.columns = ['Mention', 'Frequency']
print("\nMention Counts:\n", mention_counts)

# Plot mentions
plt.figure(figsize=(10,6))
sns.barplot(data=mention_counts.head(5), x='Mention', y='Frequency')
plt.title("Top Mentioned Users")
plt.xticks(rotation=45)
plt.savefig("mentions_chart.png")
plt.show()

# ========================
# PART 2: LOCATION ANALYSIS WITH MAP
# ========================
data_reviews = {
    'Review': [
        "Great product, loved it! (New York)",
        "Highly recommended, excellent quality (Los Angeles)",
        "Good value for money (Chicago)",
        "Not satisfied with the product (Houston)",
        "Amazing experience, would buy again (Phoenix)",
        "Very happy with the purchase (New York)",
        "Top-notch service, will buy again (Chicago)",
        "Disappointed with the quality (New York)"
    ]
}
df = pd.DataFrame(data_reviews)
```

```
# Extract location
df['Location'] = df['Review'].apply(lambda x: x.split('(')[-1].strip(')').strip())

# Initialize geocoder
geolocator = Nominatim(user_agent="geo_app")

def geocode_location(loc):
    try:
        location = geolocator.geocode(loc)
        if location:
            return location.latitude, location.longitude
    except:
        pass
    return None, None

df['Latitude'], df['Longitude'] = zip(*df['Location'].apply(geocode_location))

# Remove nulls
df = df.dropna(subset=['Latitude', 'Longitude'])
print("\nGeocoded Data:\n", df[['Review','Location','Latitude','Longitude']])

# ========================
# GEO DATAFRAME (MAP)
# ========================
gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df['Longitude'], df['Latitude'])
)

# Load world map
world = gpd.read_file(
```

```
    "https://naturalearth.s3.amazonaws.com/110m_cultural/ne_110m_admin_0_countries.zip"
)

# Plot map
fig, ax = plt.subplots(figsize=(10,6))
world.plot(ax=ax, color='lightgrey', edgecolor='black')
gdf.plot(ax=ax, marker='o', color='blue', markersize=100, alpha=0.6)
plt.title("Product Reviews by Location (World Map)")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.savefig("world_map.png")
plt.show()

# ========================
# ADDITIONAL BAR CHART
# ========================
location_counts = df['Location'].value_counts().reset_index()
location_counts.columns = ['Location', 'Review Count']
plt.figure(figsize=(10,6))
sns.barplot(data=location_counts, x='Location', y='Review Count')
plt.title("Reviews per Location")
plt.xticks(rotation=45)
plt.savefig("location_bar_chart.png")
plt.show()
```

EXERCISE-1 (Under EXP-8)

Python

```
# Install libraries
!pip install pandas matplotlib

# ========================
# HASHTAG EXTRACTION & TREND ANALYSIS (VS CODE VERSION)
```

```
# ========================
import pandas as pd
import matplotlib.pyplot as plt
import re
from collections import Counter

# ========================
# CREATE SAMPLE DATA
# ========================
data = {
    "text": [
        "Loving the new launch! #Apple #iPhone #Tech",
        "Great performance by the team #Sports #Win",
        "New trends in AI are amazing #AI #Tech #Innovation",
        "Workout time! #Fitness #Health #Gym",
        "Latest iPhone review is out #Apple #iPhone #Review",
        "AI is the future #AI #MachineLearning #Tech",
        "Stay healthy and fit #Health #Fitness",
        "Big match tonight #Sports #Excited",
        "Technology is evolving fast #Tech #Innovation",
        "Gym life is the best #Fitness #Gym"
    ]
}
df = pd.DataFrame(data)
print("\nDataset:\n", df)

# ========================
# EXTRACT HASHTAGS
# ========================
def extract_hashtags(text):
    return re.findall(r"#\w+", text)
```

```
df['hashtags'] = df['text'].apply(extract_hashtags)

# Flatten hashtags
all_hashtags = [tag.lower() for tags in df['hashtags'] for tag in tags]

# ========================
# FREQUENCY ANALYSIS
# ========================
hashtag_counts = Counter(all_hashtags)
print("\nTop Trending Hashtags:")
for tag, count in hashtag_counts.most_common(10):
    print(tag, ":", count)

# ========================
# VISUALIZATION
# ========================
top_tags = dict(hashtag_counts.most_common(10))
plt.figure(figsize=(10,5))
plt.bar(top_tags.keys(), top_tags.values())
plt.xticks(rotation=45)
plt.title("Top Trending Hashtags")
plt.xlabel("Hashtags")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("hashtag_trends.png")
plt.show()
```

EXERCISE-2 (Under EXP-8)

Python

```
# Install libraries
!pip install pandas matplotlib textblob
```

```
# ========================
# SENTIMENT ANALYSIS ACROSS LOCATIONS (VS CODE VERSION)
# ========================
import pandas as pd
import matplotlib.pyplot as plt
from textblob import TextBlob

# ========================
# CREATE SAMPLE DATASET
# ========================
data = {
    "location": [
        "Mumbai","Mumbai","Delhi","Delhi","Bangalore","Bangalore",
        "Chennai","Chennai","Pune","Pune"
    ],
    "review": [
        "Amazing product and great quality",
        "Very satisfied with the service",
        "Not happy with the product",
        "Average experience",
        "Excellent performance and design",
        "Loved it, totally worth it",
        "Bad quality and poor service",
        "Not worth the price",
        "Good value for money",
        "Decent but can improve"
    ]
}
df = pd.DataFrame(data)
print("\nDataset:\n", df)
```

```
# ========================
# SENTIMENT ANALYSIS
# ========================
def get_sentiment(text):
    return TextBlob(str(text)).sentiment.polarity

df['sentiment_score'] = df['review'].apply(get_sentiment)

# Classify sentiment
def classify(score):
    if score > 0:
        return "Positive"
    elif score < 0:
        return "Negative"
    else:
        return "Neutral"

df['sentiment'] = df['sentiment_score'].apply(classify)
print("\nSentiment Data:\n", df)

# ========================
# SENTIMENT DISTRIBUTION
# ========================
sentiment_dist = pd.crosstab(df['location'], df['sentiment'])
print("\nSentiment Distribution by Location:\n", sentiment_dist)

# ========================
# VISUALIZATION
# ========================
plt.figure(figsize=(10,6))
```

```
sentiment_dist.plot(kind='bar', stacked=True)
plt.title("Sentiment Distribution Across Locations")
plt.xlabel("Location")
plt.ylabel("Number of Reviews")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("sentiment_location.png")
plt.show()
```